CERTIFYING THE JUDGE: FALSIFIABLE PROPERTIES FOR LLM-BASED EVALUATION OF FORMAL CODE

Anonymous authors
Paper under double-blind review

ABSTRACT

Large language models judge whether AI-written formal specifications express the right guarantees, a question that running tests or checking proofs cannot settle. Reliability is usually assessed through agreement with human ratings, leaving each new judge configuration dependent on human validation. We propose collecting labels once, then screening new judges with falsifiable properties that specify how scores should respond to controlled edits. Four properties target identity, bug monotonicity, missing-specification monotonicity, and stability for Lean 4 specifications. A fixed equal-weight index, $\mathrm{TI}_{\mathrm{var}}$, summarizes bug monotonicity and variance. We compare three judge families with eight configurations each on a 72-pair, single-annotator validation set that also informed protocol selection. For Codex, index–human-agreement Spearman correlations are 0.833 on averaged labels and 0.905 and 0.738 on the two rating passes. Bug-ladder reversals expose a lexical grader that human agreement alone would favor, although the scalar index still scores it highly. Judge validation thus moves toward a one-time human panel plus automated, falsifiable checks that each new judge must face.

1 INTRODUCTION

AI agents write both programs and the checks intended to establish their correctness, including tests (Haque et al., 2026) and formal specifications (Yang et al., 2024). Running a generated test reveals whether the program and test agree, not whether either expresses the intended behavior. This is an instance of the test-oracle problem: deciding which behavior should count as correct (Barr et al., 2015). A formal specification states the guarantees a program should satisfy, such as returning a sorted permutation of its input. Its role resembles that of a grading test, but a universally quantified theorem covers every input rather than sampled cases (Claessen & Hughes, 2000). Lean can check a proof of that theorem; it cannot establish that the author chose the right theorem (Lahiri, 2024).

Large language models (LLMs) already help replace the human oracle, filtering autoformalized statements in theorem-proving pipelines (Lin et al., 2025; Wang et al., 2025). Human agreement is a common criterion for validating these judges (Liu et al., 2023; Zheng et al., 2023). Agreement varies across models, tasks, and evaluated properties, so evidence for one configuration does not establish another's reliability (Bavaresco et al., 2025). Existing labels can assess another judge on the same examples without fresh annotation. However, repeatedly selecting judges against those labels risks adaptive overfitting (Dwork et al., 2015). New artifact distributions also raise a separate question about whether those labels remain representative.

We propose using one human panel to assess a property suite, then screening subsequent judge configurations without consulting those labels. The screen asks whether a judge responds appropriately when a controlled edit preserves or damages a reference artifact. A failed property identifies a concrete behavior that human agreement alone can hide. Passing the screen supplies evidence about the tested behaviors, without guaranteeing agreement on unseen artifacts.

We instantiate this protocol for Lean 4 formal specifications judged against gold references (Figure 1). Four properties test identity, responses to cumulative bugs, responses to removed theorem statements and unit tests, and score stability. A fixed equal-weight index, $\mathrm{TI}_{\mathrm{var}}$, summarizes bug monotonicity and two variance penalties. The full diagnostic suite remains necessary because this scalar omits some properties.

On a 72-pair validation set, the index tracks judge-level human agreement across three families, each containing eight configurations. The Codex association is Spearman 0.833, with a 95% bootstrap interval of $[0.24, 1.00]$. These estimates are optimistic because the same labels informed the index form, prompts, aggregation, and threshold. A deliberately lexical judge agrees with humans yet reverses its scores on most bug ladders.

The aim is to amortize annotation through reusable checks, with transfer to new judges and domains assessed separately. Our contributions are:

- A protocol that combines a one-time human validation set with four falsifiable properties for screening judge configurations without new labels (Section 4).
- An evaluation linking committed scores to validation-set association, lexical failures, and the limits of reducing the property suite to one index (Section 5.3).

2 Related Work

LLM-as-a-judge validation. G-Eval (Liu et al., 2023) and MT-Bench (Zheng et al., 2023) compare judges with human ratings, leaving reliability dependent on the evaluated tasks and configurations. Bavaresco et al. (Bavaresco et al., 2025) document this dependence across tasks rather than establishing a universal judge. Schroeder and Wood-Doughty (Schroeder & Wood-Doughty, 2024) measure consistency across repeated judgments, which does not by itself establish correctness. Our protocol retains human agreement as a validation criterion and adds tests with specified directions of response.

Judges for code. ICE-Score (Zhuo, 2024) and CodeJudge (Tong & Zhang, 2024) use models to assess generated code where execution tests provide incomplete evidence, but their human-agreement results do not certify later judge configurations. SWT-Bench validates generated tests against developers' fixes (Mündler et al., 2024), requiring a reference behavior rather than treating test execution as proof of test validity. Our controlled perturbations serve a related purpose for judges: they specify how a score should change without labeling each perturbed artifact.

Judging formal statements. FormalAlign learns alignment between informal and formal statements (Lu et al., 2025), whereas our target is the behavior of a judge comparing candidate and gold specifications. ReForm validates formal-statement judges against expert annotations (Chen et al., 2026), without establishing that later configurations satisfy our behavioral properties. VeriBench evaluates agent-generated Lean artifacts (Miranda et al., 2026); we study the judges used to score their formal specifications. These tasks require checking what a statement means, beyond whether a proof checker accepts it.

3 Problem Setup

Intent, specification, and artifact. An item contains an informal problem statement, a gold Lean 4 artifact g, and a candidate artifact x generated by an agent. We call the informal requirement the *intent*. An artifact is a Lean source file containing an implementation and, potentially, theorem statements, predicates, unit tests, and proofs. Its *formal specification* consists of the theorem statements about the implementation together with the predicates they use. For example, a sorting specification can require both ordered output and preservation of the input elements. A proof certifies a theorem under its stated assumptions; it does not determine whether those requirements capture the intent. A unit test checks a concrete case and is not part of the specification under our terminology (Appendix F). We use "specification" in this sense regardless of who wrote it.

Reference-based specification quality. The target is how completely and correctly the candidate specification captures the gold reference. A candidate can type-check yet omit a requirement, impose an unintended assumption, or assert a vacuous property. The judge prompts ask which reference guarantees the candidate captures and assign a score of zero to trivial statements such as `True` (Appendix A). This scoring instruction does not establish that judges detect every such failure. A gold reference supplies a concrete comparison target; it does not make the judge immune to strategic manipulation (Robertson & Koyejo, 2025).

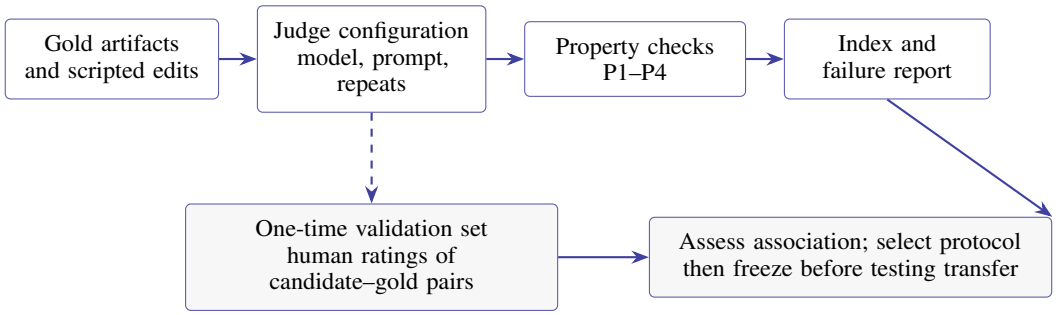


Figure 1: The human panel informs the protocol once; later judges face the upper sequence of automated checks. The lower sequence measures validation-set association and selects protocol choices, so its results are not a held-out test.

Stage	Human labels	Judge computation
Establish protocol	Rate candidate–gold pairs	Measure human agreement and property behavior
Select protocol	Use validation ratings	Choose index form, prompts, aggregation, threshold
Screen new judge	Do not consult labels	Run frozen probes; inspect index and failures
Test transfer	Collect independent ratings	Evaluate frozen choices on untouched items

Table 1: Screening a new judge requires inference but no new labels; establishing transfer requires a separate test.

Judge configuration. We define $\theta = (\text{model}, \text{prompt}, n_{\text{repeats}}, \text{aggregation})$ and write its score as $J_\theta(x, g) \in [0, 1]$. The headline panel uses a median over repeated calls. Despite the prompt label "REFERENCE IMPLEMENTATION," its reference field contains the gold file's final theorem block; its candidate field contains the full candidate file. Thus the measured task depends on this extraction rule, not only on the nominal rubric. A change to that rule requires a new assessment. We distinguish a judge's agreement with artifact ratings from the association, across judge configurations, between that agreement and a property index.

4 Property Certification Protocol

A one-time panel and reusable checks. Human ratings link judge behavior to the intended scoring task on a fixed validation set. For each configuration, we measure agreement with those ratings and evaluate scripted perturbations of gold artifacts. The perturbations require judge calls but no additional human labels. We use the validation set to select the index form, prompts, repeat aggregation, and decision threshold. After freezing these choices, subsequent configurations can face the same property checks without accessing the labels (Figure 1). This is a screening protocol; the present evidence does not establish transfer to an unseen configuration or distribution.

Four falsifiable properties. **P1 (identity)** requires the maximum score when an artifact is compared with itself, $J_\theta(g, g) = 1$. **P2 (bug monotonicity)** requires non-increasing scores as errors accumulate. The historical bug ladder used here alters the candidate text, including theorem statements; it is not restricted to implementation bugs. **P3 (missing-specification monotonicity)** requires non-increasing scores as theorem statements and unit tests are removed. We retain this historical property name although unit tests are not specifications under Section 3. Each ladder extends the preceding level, so its expected ordering concerns cumulative damage rather than equal severity of individual edits. Ties are allowed; a local score increase is a reversal. **P4 (stability)** requires stable scores across repeated calls and meaning-preserving rewrites. These include alpha-renaming, which consistently changes bound-variable names without changing their meaning. The properties are targets for the probes; they are not theorems establishing the validity of every scripted transformation.

A fixed equal-weight summary. The protocol retains individual diagnostics because different failures need different checks. Our primary scalar, $\mathrm{TI}_{\mathrm{var}}$, uses only bug-ladder scores, combining their average monotonicity with two variance penalties:

$$\mathrm{TI}_{\mathrm{var}} = \exp\left[\tfrac{1}{3}\log C_f + \tfrac{1}{3}\log(1-\widetilde{V}_c) + \tfrac{1}{3}\log(1-\widetilde{V}_s)\right]. \tag{1}$$

Here C_f is the mean transformed per-file Spearman correlation between bug level and score. V_c is the variance of these per-file monotonicity values, and V_s is the mean within-level variance across repeated calls. The tildes denote min–max normalization across the configurations in the same family panel. The implementation floors each logarithm's argument at 10^{-6}. Appendix B specifies aggregation, normalization, and undefined-correlation handling.

Multiplication prevents a strong component from fully offsetting a weak one; equal weights avoid fitting component weights to the human ratings. The scalar nevertheless omits P1, P3, rewrite invariance, and a direct penalty for local reversals. It therefore summarizes selected behavior rather than replacing the property report. Moreover, panel-dependent normalization means that its absolute value can change when the comparison panel changes. Using it prospectively requires a frozen normalization reference, whose transfer we have not tested.

5 Experiments

5.1 Data and human panel

We evaluate candidate–gold pairs from VeriBench (Miranda et al., 2026). One annotator supplied holistic ratings on a 0–5 scale in two passes, pass1 with a stricter rubric and pass2 with a more lenient rubric. The collection covered 75 pairs from 25 problems with three agent generations per problem. One gold filename could not be resolved, leaving 72 analyzed pairs from 24 problems. We report each pass and the archived averaged-label view; they are two judgments by one person, not independent annotators. Inter-pass agreement is Spearman 0.249, quadratic-weighted $\kappa = 0.372$, and mean absolute error 0.771 on the shared pairs (Appendix D). These statistics describe label disagreement; they do not establish an upper bound on judge agreement. The original written rubric was not recovered, so we cannot reconstruct every annotation instruction.

This panel is the *validation set*: it informed index-form selection, repeat aggregation, the threshold, and the prompt panel, so correlations on it are optimistic. The fixed equal-weight $\mathrm{TI}_{\mathrm{var}}$ has no training set. A separate weight-tuned comparison uses these same pairs, and no analysis constitutes a test-set result. For each judge, artifact-level agreement is the Spearman correlation between its scores and the human ratings on the same analyzed pairs. We then ask whether the index orders configurations by that agreement.

5.2 Property checks on GPT-5.3-Codex

Figure 2 summarizes the 24-problem panel for three Codex prompts. Their mean scores decrease as bug-ladder edits accumulate, but averages conceal local reversals. The strict prompt reverses on four of 24 ladders, whereas the baseline and checklist prompts have no reversals (Table 3). The archived alpha probes produce the largest score changes among the four rewrite types, but their generator can produce malformed Lean text. The baseline and checklist prompts assign them zero mean score, despite near-maximum scores for the other rewrites. The strict prompt gives alpha-renamed artifacts a mean score of 0.179. Appendix E retains the per-configuration diagnostics and the corresponding Claude Opus summary.

Alpha-renaming and a canonicalization proxy. The historical alpha routine can duplicate the assignment marker and rename signature variables without updating the proof body. Its score drops therefore cannot establish a failure of invariance under valid alpha-renaming. A post-hoc proxy replaces each judge's alpha scores with its median score under style edits. Across the eight Codex-panel configurations, this lowers the mean per-judge standard deviation across identity-probe scores from 0.250 to 0.070. This manipulation changes recorded scores rather than the judge's inputs. A real fix must rescore canonicalized text and verify that the transformation preserves meaning.

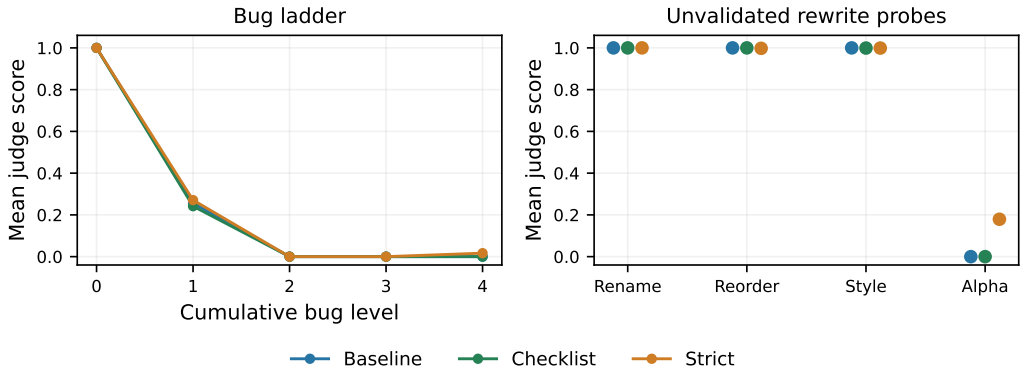


Figure 2: Mean scores fall with cumulative bug edits, but malformed alpha probes sharply change the Codex judges' scores. Each point averages median scores over 24 problems, with three calls per problem and prompt; these are descriptive means, not uncertainty intervals. Local reversals remain visible in Table 3.

	Averaged labels				$\mathrm{TI}_{\mathrm{var}}$ by pass	
Family	$v1$	$v2$	$\mathrm{GM}_{.4,.6}$	$\mathrm{TI}_{\mathrm{var}}$	pass1	pass2
Codex	0.762	0.476	0.857	0.833	0.905	0.738
Claude Opus	0.429	0.143	0.714	0.643	0.810	0.595
Qwen3-Coder	0.524	0.500	0.786	0.905	0.476	0.857
Codex $\mathrm{TI}_{\mathrm{var}}$: 95% interval				$[0.24, 1.00]$	$[0.41, 1.00]$	$[0.11, 1.00]$
Codex $\mathrm{TI}_{\mathrm{var}}$: permutation p				0.017	0.005	0.045

Table 2: The fixed index is competitive with a two-component geometric comparison whose weights were selected on pass1. Entries are Spearman correlations between each index and judge-level human agreement on the validation set, with eight configurations per family. Codex intervals and permutation probabilities use 10,000 resamples and permutations, seed 42; corresponding intervals for the other families are not reported. Appendix B defines all indices.

Coverage of P3. The eight-configuration panel contains identity rewrites and bug ladders but no missing-specification ladders. Consequently, it cannot establish which association P3 has with human agreement. We retain P3 as a proposed part of the protocol, without attributing the bug-ladder statistics below to it.

5.3 The index tracks judge-level human agreement on the validation set

The three primary families are GPT-5.3-Codex, Claude Opus 4.7, and Qwen3-Coder. Each eight-configuration panel contains three semantic prompts, a deliberately lexical prompt on the same model, two GPT-4o-mini prompts, edit distance, and a random control. These are family panels with shared controls, not eight independent models per family. The frozen model identifiers and complete main-panel prompts appear in Appendix A. Scores are medians over three repeated calls for language-model judges; Codex-panel controls use nine repeats, and the other panels use three.

Figure 3 uses the fixed equal-weight index in Equation 1. Its judge-level Spearman association is positive for each primary family and label view (Table 2). For Codex, averaged-label Pearson and Kendall associations are 0.868 and 0.643, respectively. The Codex bootstrap intervals are wide, and the resampling does not correct for the earlier protocol selection. Pass-to-pass differences also matter: Qwen's Spearman association changes from 0.476 on pass1 to 0.857 on pass2. With eight configurations and disagreeing label passes, these estimates support a descriptive association rather than a precise or portable effect size.

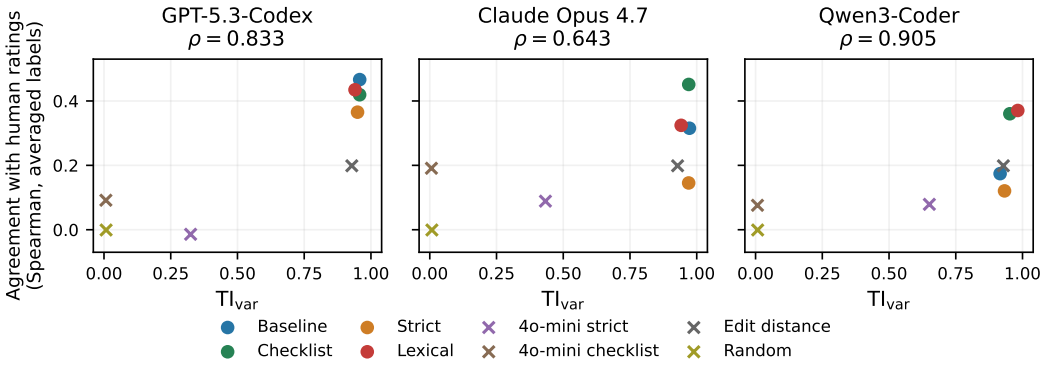


Figure 3: The fixed equal-weight index has a positive association with human agreement in each primary family, but high index values do not rule out lexical judges. Each point is one configuration; the vertical axis is its Spearman agreement with averaged human labels on 72 pairs. Panel titles report the correlation across eight configurations; Table 2 gives Codex uncertainty.

Codex-panel configuration	Human ρ (avg)	TI_{var}	Ladders with reversals
Baseline prompt	0.466	0.958	0/24
Checklist prompt	0.419	0.958	0/24
Strict prompt	0.365	0.950	4/24
Lexical prompt	0.435	0.940	20/24
GPT-4o-mini, strict	−0.014	0.324	18/24
GPT-4o-mini, checklist	0.092	0.007	16/24
Edit distance	0.199	0.928	24/24
Random	−0.001	0.008	24/24

Table 3: The lexical judge agrees with humans despite frequent bug-ladder reversals, and the scalar index penalizes it only slightly. A reversal is any local score increase as cumulative edits are added; the diagnostic uses the recorded repeat-aggregated scores.

Human agreement alone can miss a bad judge. The lexical prompt, `BAD_STRINGY`, explicitly instructs the model to ignore theorem meaning and grade textual overlap. On the Codex panel, its human agreement of 0.435 lies among the semantic prompts' values, 0.365–0.466. Yet it reverses on 20 of 24 bug ladders (Table 3). The index ranks it below the three semantic prompts, but its value remains 0.940; edit distance also scores highly at 0.928. In the Qwen panel, the lexical configuration has the highest index value, 0.982. The failure report therefore carries information that both human agreement and the scalar can miss. No validated cutoff on TI_{var} establishes that a judge is acceptable.

Choice of index. The two-component geometric comparison is numerically higher than TI_{var} for Codex and Opus on averaged labels (Table 2). It uses bug monotonicity and decision stability with weights 0.4 and 0.6, selected on pass1, rather than fixed equal weights. We retain TI_{var} for its explicit variance penalties and positive associations across the primary families and passes, not because these data establish its superiority. A separate search over six component weights gives cross-pass associations of 0.810 and 0.786 on the same pairs (Appendix C). The pass1-selected geometric form reaches 0.769 on an expanded 13-configuration panel, with a 95% interval of $[0.34, 0.95]$ (Appendix D.2). Neither comparison provides a held-out estimate. A separate Leanstral panel has averaged-label association −0.095 for the fixed index (Appendix D.3), so the positive result does not hold for every recorded family.

Weight and normalization sensitivity. Recomputing the fixed index on a local grid of weight triples gives averaged-label Spearman ranges of 0.738–0.857 for Codex, 0.643–0.714 for Opus, and 0.881–0.976 for Qwen. This analysis uses only committed component vectors and keeps their normalization fixed (Appendix C.3). It does not assess sensitivity to changing the normalization panel.

Measured diagnostic	pass1	pass2	avg
Mean identity-rewrite score	−0.21 (0.62)	0.02 (0.98)	−0.02 (0.98)
Bug-rank monotonicity	0.60 (0.13)	0.67 (0.08)	0.71 (0.06)
Bug weak monotonicity	0.63 (0.11)	0.78 (0.03)	0.75 (0.04)
Repeat decision stability	0.47 (0.24)	−0.23 (0.57)	0.14 (0.76)

Table 4: Bug weak monotonicity has the largest observed association among the measured diagnostics; this table contains no P3 missing-specification result. Entries are Spearman correlations with human agreement across eight Codex-panel configurations; unadjusted permutation probabilities appear in parentheses.

We do not report leave-one-family-out transfer: the panels share controls and normalize separately, so pooling their scores would not constitute an independent family test.

Which measured statistics matter. Among the recorded diagnostics, weak monotonicity on bug ladders has the largest association with human agreement on each label view (Table 4). It measures the absence of local increases; bug-rank correlation measures the overall direction. The archived field name `P3_weak_mono` refers to this bug statistic, not to the P3 missing-specification property. Identity-rewrite scores and repeat decision stability have weaker or inconsistent associations. Only the weak-monotonicity statistic on pass2 and averaged labels has an unadjusted permutation $p < 0.05$; these component comparisons are exploratory.

Artifact-score threshold. A historical sweep of the artifact-score threshold from 0.60 to 0.85 lowers the observed decision-flip rate from 0.058 to 0.046 and pass rate from 0.583 to 0.536 (Appendix Table 6). The selected threshold is 0.85, which has the lowest observed flip rate in that sweep. This thresholds $J_\theta(x, g)$, not the judge-level index, and measures repeatability rather than correctness of the accepted artifacts.

6 Limitations and Held-Out Test Protocol

Validation evidence and a different human criterion. The same labels informed the index form, repeat aggregation, threshold, and prompt panel, so the reported correlations are optimistic validation-set estimates. They involve one annotator and eight configurations per family, with shared controls rather than independent samples of eight models. No test-set result establishes reliability for a new judge or artifact distribution. We also collected 343 labels from five raters over 78 candidate–gold pairs. That pool uses statement coverage with a theorem-spam deduction, a different rubric from the panel's holistic ratings. Its Spearman agreement with panel labels is approximately 0.13 on 63 shared pairs.

Two pre-registered screens asked whether property indices tracked agreement with that pool. The screen of 14 configurations was underpowered; the expanded screen of 32 found no index meeting its registered criterion. Its largest absolute Spearman association was -0.39 with Holm-adjusted permutation $p = 0.25$; leave-one-out selection transfer yielded $\rho = 0.24$, $p = 0.49$. These negative results do not demonstrate transfer under the original criterion, nor can the pool serve as a test set for that criterion. Appendix D.3 records the corresponding negative-control evidence.

Coverage of the screen. The probes do not establish sensitivity to vacuity, subtle weakening, over-constraint, or formatting-only edits beyond the four rewrites studied here. Constructing probes with known directions for these failures is the next methodological step; we claim no result for them. The historical alpha generator can produce malformed text, so its low scores do not establish sensitivity to valid alpha-renaming. Other scripted rewrites and the intended ordering of bug edits also lack semantic validation. We cannot separate rubric ambiguity from misunderstanding Lean semantics with the current annotations. Presentation, including redundant theorems, implementation bloat, and unnecessary abstraction, is unassessed; holistic human ratings may reflect it implicitly. We did not optimize candidates against the judge, so resistance to gaming is untested.

All candidates and probes come from VERIBENCH in Lean 4. The protocol could be instantiated for another proof assistant, but neither cross-assistant nor cross-benchmark transfer has been tested. A score is relative to the supplied gold reference and extraction rule, not a guarantee that all intended requirements are present. Reference-free screening and the number of annotators needed for a stable criterion remain open. The index also depends on its comparison panel and omits constant ladders with undefined correlations; these implementation choices limit portable screening.

A locked test. A decisive test would first freeze the rubric, property generators, score extraction, aggregation, index, normalization reference, and any rejection rule. Independent raters would then label new candidate–gold pairs under that rubric, with new judge configurations fixed before their labels are examined. The analysis would report both artifact-level agreement and judge-level association, including uncertainty and property-specific failures. A separate distribution-shift test would be needed for new domains. Because the original rubric text is unavailable, recovering it or validating an explicitly reconstructed criterion is a prerequisite to claiming a matched-rubric test.

7 CONCLUSION

A one-time human panel can support a protocol that screens new judges through falsifiable behavioral checks without collecting labels for each configuration. On the present validation set, the fixed equal-weight index is associated with human agreement, but its high scores can coexist with lexical failures. Individual property diagnostics therefore remain essential, and a locked test must establish transfer beyond the labels used to select the protocol. The goal is to make human judgment a reusable foundation for checking new judges, with each judge accountable to explicit tests of its behavior.

Reproducibility Statement

The accompanying repository preserves frozen scores and summaries under `experiments/archive/`. Appendix A gives the prompt templates, exact model identifiers, input extraction, and repeat aggregation used for the main panels. Appendix B specifies the indices, including normalization and numerical edge cases. The manuscript evidence script, `experiments/34_iclr_mega_rewrite_astra/analysis/rebuild_paper_evidence.py`, reads committed artifacts, verifies their hashes, and regenerates the corrected figures and sensitivity summaries without inference. Its output manifest maps every input to a repository path and content hash. The adjacent report maps every reported table to the archived script and machine-readable output that produced it. The original human-panel rubric text was not recovered, which limits exact replication of the annotation process. We describe repository paths without an identifying repository URL for anonymous review.

Ethics Statement

An author supplied the human ratings used for the headline validation analysis. The study evaluates generated Lean code and formal statements, without personal data in the judged artifacts. It makes no deployment claim and does not establish that screened judges can replace expert review in consequential settings.

LLM Usage Statement

LLMs served as the judges under study and assisted with writing and editing under author supervision. The authors are responsible for the manuscript's claims, citations, and interpretation of the recorded results.

References

Earl T. Barr, Mark Harman, Phil McMinn, Muzammil Shahbaz, and Shin Yoo. The oracle problem in software testing: A survey. *IEEE Transactions on Software Engineering*, 41(5):507–525, 2015. doi: 10.1109/TSE.2014.2372785.

Anna Bavaresco, Raffaella Bernardi, Leonardo Bertolazzi, Desmond Elliott, Raquel Fernández, Albert Gatt, Esam Ghaleb, Mario Giulianelli, Michael Hanna, Alexander Koller, André F. T. Martins, Philipp Mondorf, Vera Neplenbroek, Sandro Pezzelle, Barbara Plank, David Schlangen, Alessandro Suglia, Aditya K. Surikuchi, Ece Takmaz, and Alberto Testoni. Llms instead of human judges? a large scale empirical study across 20 nlp evaluation tasks. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 2: Short Papers)*, pp. 238–255. Association for Computational Linguistics, 2025. doi: 10.18653/v1/2025.acl-short.20. URL https://aclanthology.org/2025.acl-short.20/.

Guoxin Chen, Jing Wu, Xinjie Chen, Wayne Xin Zhao, Ruihua Song, Chengxi Li, Kai Fan, Dayiheng Liu, and Minpeng Liao. ReForm: Reflective autoformalization with prospective bounded sequence optimization. In *International Conference on Learning Representations (ICLR)*, 2026. URL https://proceedings.iclr.cc/paper_files/paper/2026/hash/419bd6a89ab56e9c09ededb560b06192-Abstract-Conference.html.

Koen Claessen and John Hughes. QuickCheck: A lightweight tool for random testing of Haskell programs. In *Proceedings of the Fifth ACM SIGPLAN International Conference on Functional Programming (ICFP)*, pp. 268–279, 2000. doi: 10.1145/351240.351266.

Cynthia Dwork, Vitaly Feldman, Moritz Hardt, Toniann Pitassi, Omer Reingold, and Aaron Roth. The reusable holdout: Preserving validity in adaptive data analysis. *Science*, 349(6248):636–638, 2015. doi: 10.1126/science.aaa9375. URL https://doi.org/10.1126/science.aaa9375.

Sabrina Haque, Sarvesh Ingale, and Christoph Csallner. Do autonomous agents contribute test code? a study of tests in agentic pull requests, 2026. URL https://arxiv.org/abs/2601.03556.

Shuvendu K. Lahiri. Evaluating LLM-driven user-intent formalization for verification-aware languages. In *Proceedings of the 24th Conference on Formal Methods in Computer-Aided Design (FMCAD)*, pp. 142–147. TU Wien Academic Press, 2024. doi: 10.34727/2024/isbn.978-3-85448-065-5_19. URL https://arxiv.org/abs/2406.09757.

Yong Lin, Shange Tang, Bohan Lyu, Jiayun Wu, Hongzhou Lin, Kaiyu Yang, Jia Li, Mengzhou Xia, Danqi Chen, Sanjeev Arora, and Chi Jin. Goedel-prover: A frontier model for open-source automated theorem proving, 2025. URL https://arxiv.org/abs/2502.07640.

Yang Liu, Dan Iter, Yichong Xu, Shuohang Wang, Ruochen Xu, and Chenguang Zhu. G-eval: Nlg evaluation using gpt-4 with better human alignment. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, pp. 2511–2522. Association for Computational Linguistics, 2023. doi: 10.18653/v1/2023.emnlp-main.153. URL https://arxiv.org/abs/2303.16634.

Jianqiao Lu, Yingjia Wan, Yinya Huang, Jing Xiong, Zhengying Liu, and Zhijiang Guo. FormalAlign: Automated alignment evaluation for autoformalization. In *International Conference on Learning Representations*, 2025. URL https://proceedings.iclr.cc/paper_files/paper/2025/hash/fceedf8c9c0ff51f41b9fe0294ef0070-Abstract-Conference.html.

Brando Miranda, Srivatsava Daruru, Ethan S Hersch, Zhanke Zhou, Allen Nie, Daneshvar Amrollahi, Leni Aniva, Iddah Mlauzi, Kirill Acharya, Elyas Obbad, Dilara Soylu, Weston Kirk, Zixiao Jolene Wang, Kai Fronsdal, Ying Li, Donald Poindexter Jr, Rakshit Kaushik, Shurui Liu, Yegor Denisov-Blanch, Steven Dillmann, Simon Obstbaum, Santiago Cuellar, John Sarracino, Rylan Schaeffer, Mo Tiwari, Donghyun Lee, Bo Han, and Sanmi Koyejo. Veribench: An end-to-end formal verification benchmark for AI coding agents in lean 4. In *3rd AI for Math Workshop: Toward Self-Evolving Scientific Agents*, 2026. URL https://openreview.net/forum?id=lkL0qnUv3p.

Niels Mündler, Mark Niklas Müller, Jingxuan He, and Martin Vechev. SWT-bench: Testing and validating real-world bug-fixes with code agents. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2024. URL https://arxiv.org/abs/2406.12952.

Zachary Robertson and Sanmi Koyejo. Let's measure information step-by-step: AI-based evaluation beyond vibes, 2025. URL https://arxiv.org/abs/2508.05469.

Kayla Schroeder and Zach Wood-Doughty. Can you trust llm judgments? reliability of llm-as-a-judge, 2024. URL https://arxiv.org/abs/2412.12509.

Weixi Tong and Tianyi Zhang. Codejudge: Evaluating code generation with large language models. In *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pp. 20032–20051. Association for Computational Linguistics, 2024. doi: 10.18653/v1/2024.emnlp-main.1118. URL https://arxiv.org/abs/2410.02184.

Haiming Wang, Mert Unsal, Xiaohan Lin, Mantas Baksys, Junqi Liu, Marco Dos Santos, Flood Sung, Marina Vinyes, Zhenzhe Ying, Zekai Zhu, Jianqiao Lu, Hugues de Saxcé, Bolton Bailey, Chendong Song, Chenjun Xiao, Dehao Zhang, Ebony Zhang, Frederick Pu, Han Zhu, Jiawei Liu, Jonas Bayer, Julien Michel, Longhui Yu, Léo Dreyfus-Schmidt, Lewis Tunstall, Luigi Pagani, Moreira Machado, Pauline Bourigault, Ran Wang, Stanislas Polu, Thibaut Barroyer, Wen-Ding Li, Yazhe Niu, Yann Fleureau, Yangyang Hu, Zhouliang Yu, Zihan Wang, Zhilin Yang, Zhengying Liu, and Jia Li. Kimina-prover preview: Towards large formal reasoning models with reinforcement learning, 2025. URL https://arxiv.org/abs/2504.11354.

Kaiyu Yang, Gabriel Poesia, Jingxuan He, Wenda Li, Kristin Lauter, Swarat Chaudhuri, and Dawn Song. Formal mathematical reasoning: A new frontier in AI, 2024. URL https://arxiv.org/abs/2412.16075.

Lianmin Zheng, Wei-Lin Chiang, Ying Sheng, Siyuan Zhuang, Zhanghao Wu, Yonghao Zhuang, Zi Lin, Zhuohan Li, Dacheng Li, Eric P. Xing, Hao Zhang, Joseph E. Gonzalez, and Ion Stoica. Judging llm-as-a-judge with mt-bench and chatbot arena. In *Advances in Neural Information Processing Systems*, volume 36, pp. 46595–46623. Curran Associates, Inc., 2023. doi: 10.52202/075280-2020. URL https://arxiv.org/abs/2306.05685.

Terry Yue Zhuo. ICE-score: Instructing large language models to evaluate code. In Yvette Graham and Matthew Purver (eds.), *Findings of the Association for Computational Linguistics: EACL 2024*, pp. 2232–2242, St. Julian's, Malta, March 2024. Association for Computational Linguistics. doi: 10.18653/v1/2024.findings-eacl.148. URL https://aclanthology.org/2024.findings-eacl.148/.

A JUDGE PROMPTS AND SCORING CONFIGURATION

The primary family model identifiers are `gpt-5.3-codex`, `claude-opus-4-7`, and `Qwen/Qwen3-Coder-30B-A3B-Instruct`. Each family panel contains the three semantic prompts below and `BAD_STRINGY`, plus `gpt-4o-mini` with the checklist and strict prompts. The other two configurations are the deterministic edit-distance control `editdist_e1` and seeded control `random_r1`. All model prompts request scores in $[0, 1]$, not integers on a 0–10 scale. The human ratings use 0–5; correlations compare their ordering with judge scores without equating the scales.

The reference field retains its historical label "REFERENCE IMPLEMENTATION," but the code fills it with the gold's final theorem block. The candidate field contains the full candidate file for human-agreement scoring; property probes operate on the extracted theorem text. If no theorem block is found, the extractor uses the full file. The Codex panel aggregates three recorded calls for model configurations and nine for controls by the median. The Opus and Qwen panels aggregate three calls for every configuration by the median. These counts come from the saved repeat arrays; a summary-level repeat setting alone does not describe reused cached rows. The random control is deterministic for its input and seed, so repeated identical scores do not imply a reliable semantic judge. Prompt labels P1, P4, and P8 are historical identifiers and do not denote the property numbers P1–P4.

P1_BASELINE

```
You are an expert in Lean 4 formal verification.
Score how well the GENERATED THEOREM captures the correctness properties
of the REFERENCE IMPLEMENTATION. Score from 0.0 to 1.0.

- 1.0: Semantically equivalent to what the reference guarantees
- 0.7-0.9: Captures most properties, misses minor ones
- 0.4-0.6: Captures some properties, has meaningful gaps
- 0.1-0.3: Technically valid Lean but misses core properties
- 0.0: Trivially true (e.g., `True`) or incorrect

REFERENCE IMPLEMENTATION:
{reference_impl}

GENERATED THEOREM:
{generated_theorem}

Respond ONLY with valid JSON, no markdown:
{"score": <float 0.0-1.0>,
 "reasoning": "<one sentence max>"}
```

P4_CHECKLIST

```
Evaluate this in order:
1) What properties does REFERENCE IMPLEMENTATION guarantee?
2) Which are preserved by GENERATED THEOREM?
3) Are there missing core guarantees or trivialization?
4) Map to score in [0.0, 1.0]:
   1.0 equivalent; 0.7-0.9 minor gaps;
   0.4-0.6 meaningful gaps; 0.1-0.3 weak;
   0.0 trivial/incorrect.

REFERENCE IMPLEMENTATION:
{reference_impl}

GENERATED THEOREM:
{generated_theorem}

Return JSON only:
{"score": <float 0.0-1.0>,
 "reasoning": "<one sentence>"}
```

P8_STRICT_B

```
Evaluate conservatively. If uncertain, choose the lower score.
Use [0.0,1.0]:
1.0 equivalent semantics
0.7-0.9 mostly right, minor misses
0.4-0.6 substantial omissions
0.1-0.3 weak relation
0.0 trivial/incorrect
```

```
REFERENCE IMPLEMENTATION:
{reference_impl}

GENERATED THEOREM:
{generated_theorem}

Return valid JSON only:
{"score": <float>,
 "reasoning": "<one sentence>"}
```

BAD_STRINGY negative control

```
You are a text similarity grader.
Compare generated theorem text to the reference implementation text.
Do NOT evaluate semantic correctness or theorem meaning.
Score ONLY based on superficial lexical/format overlap:
- high score if tokens/phrases look similar
- low score if wording/structure differs
Reference implementation:
{reference_impl}

Generated theorem:
{generated_theorem}

Return ONLY a JSON object:
{"score": <float 0.0 to 1.0>,
 "reasoning": "<short reason>"}
```

B Index Definitions and Implementation

Fixed equal-weight index. For each file f and bug level ℓ, let $s_{f\ell}$ be the median recorded repeat score and $v_{f\ell}$ its population variance. Let ρ_f be Spearman's correlation between level and $s_{f\ell}$, and set $c_f = \mathrm{clip}((1-\rho_f)/2, 0, 1)$. Writing $\mathcal{F}$ for the included ladders, the archived implementation uses

$$C_f = \frac{1}{|\mathcal{F}|}\sum_{f\in\mathcal{F}} c_f, \qquad V_c = \mathrm{Var}_{f\in\mathcal{F}}(c_f), \qquad V_s = \frac{\sum_{f\in\mathcal{F}}\sum_{\ell} v_{f\ell}}{\sum_{f\in\mathcal{F}} L_f},$$

where L_f is the number of levels in ladder f. It skips ladders with fewer than three levels or an undefined Spearman correlation, including constant-score ladders. All primary committed vectors report 24 included ladders per configuration. If none remain, the code substitutes $C_f = 0$, $V_c = 1$, and $V_s = 1$. It min–max normalizes each variance across the current family panel; a constant variance column maps to zero. Equation 1 then applies weights $1/3$ to C_f, $1-\widetilde{V}_c$, and $1-\widetilde{V}_s$, after clipping each to $[10^{-6}, 1]$. Thus an extreme variance does not produce an exact zero index, and adding a judge can change other judges' normalized scores. The analysis preserves these choices rather than changing the frozen index after observing results.

Arithmetic comparisons. The recorded $v1$ and $v2$ are

$$v1 = 0.30\,\mathrm{ident} + 0.30\,\mathrm{mono} + 0.25\,\mathrm{weak} + 0.15\,\mathrm{stab},$$
$$v2 = 0.20\,\mathrm{ident} + 0.20\,\mathrm{mono} + 0.15\,\mathrm{weak} + 0.10\,\mathrm{stab} + 0.20\,\mathrm{anti} + 0.15\,\mathrm{subtle}.$$

Here ident is mean score over identity-rewrite probes, $\mathrm{mono} = (1-\rho_{\mathrm{bug}})/2$ uses the pooled bug-level Spearman correlation, and weak is the weak-monotonicity fraction on bug ladders. The latter is the historical field `P3_weak_mono`; it contains no missing-specification measurement. stab is one minus the fraction of scored pairs whose repeated scores cross the artifact-score threshold. anti is the complement of the absolute score correlation with normalized edit distance, clipped to $[0,1]$, and subtle is the clipped mean first-level score drop divided by 0.25. The implementation clips these components to $[0,1]$. These arithmetic indices combine different summaries from $\mathrm{TI}_{\mathrm{var}}$ and do not isolate the effect of changing only the aggregation function.

Geometric comparisons. The two-component comparison in Table 2 is $\mathrm{GM}_{.4,.6} = \mathrm{mono}^{0.4}\mathrm{stab}^{0.6}$, with the same 10^{-6} component floor. Its weights originate in the pass1 search over six components; it is not a fixed equal-weight baseline. The separate cross-pass analysis searches nonnegative weights summing to one over ident, mono, weak, stab, anti, and subtle. Table 5 identifies which pass supplies those fitted weights. Neither procedure fits the weights of $\mathrm{TI}_{\mathrm{var}}$.

C Additional Validation-Set Results

C.1 A separately weight-tuned geometric variant

The cross-pass analysis fits a geometric combination of six available components on one label pass and evaluates it on the other. The selected pass1 weights are 0.4 for bug monotonicity and 0.6 for decision stability. Both passes concern the same candidate–gold pairs from one annotator, so the second pass is not a test set. This analysis uses its own frozen label-alignment output; it is not a re-estimation of the fixed equal-weight index.

Pass supplying weights	Association on that pass	Association on other pass
pass1	0.881	0.810
pass2	0.857	0.786

Table 5: The tuned geometric variant remains positively associated across two rating passes of the same 72 pairs. Entries are judge-level Spearman correlations; the fitted weights are separate from $\mathrm{TI}_{\mathrm{var}}$.

C.2 Artifact-score threshold

The historical threshold sweep uses the strict Codex prompt on 176 files with three repeated scores, aggregated by the mean rather than the headline panels' median. It thresholds artifact scores, not the index used to compare judges. The raw bug-level ranking is independent of this threshold and remains $\rho_{\mathrm{bug}} = -0.790$ throughout the sweep. The intervals in Table 6 describe the recorded sweep; they do not establish accepted-artifact correctness.

Threshold	Flip rate (95% interval)	Pass rate (95% interval)
0.60	0.058 $[0.048, 0.069]$	0.583 $[0.559, 0.608]$
0.70	0.049 $[0.040, 0.059]$	0.559 $[0.537, 0.581]$
0.80	0.051 $[0.042, 0.061]$	0.544 $[0.523, 0.566]$
0.85	0.046 $[0.037, 0.057]$	0.536 $[0.515, 0.557]$

Table 6: A stricter artifact-score threshold slightly reduces recorded decision flips and the fraction accepted. The table shows selected points from the historical sweep; 0.85 has its lowest observed flip rate.

C.3 Local weight sensitivity

We recompute the index from committed component vectors on a simplex grid with each weight in $\{0.2, 0.3, 0.4, 0.5\}$ and weights summing to one. This yields 12 triples around the equal-weight choice. The original family-specific normalization and 10^{-6} floor remain fixed. No judge is rescored and no replacement weights are selected from this grid. The script and its complete output are named in the Reproducibility Statement.

Family	pass1 range	pass2 range	avg range
Codex	$[0.810, 0.905]$	$[0.738, 0.810]$	$[0.738, 0.857]$
Claude Opus	$[0.786, 0.810]$	$[0.571, 0.690]$	$[0.643, 0.714]$
Qwen3-Coder	$[0.357, 0.762]$	$[0.738, 0.952]$	$[0.881, 0.976]$

Table 7: The association remains positive on this local weight grid, with the widest pass1 variation for Qwen. Ranges cover judge-level Spearman correlations across the 12 weight triples; they are sensitivity ranges, not confidence intervals.

D HUMAN AGREEMENT AND ADDITIONAL JUDGE PANELS

D.1 AGREEMENT BETWEEN ANNOTATION PASSES

Table 8 reports the frozen agreement statistics on 72 shared analyzed pairs. The passes reflect different rubric strictness levels from the same annotator, rather than independent draws from a rater population. Their disagreement cannot identify a numerical ceiling on model agreement. The original rubric text remains unavailable.

Statistic	Estimate	95% bootstrap interval
Spearman correlation	0.249	$[-0.015, 0.489]$
Pearson correlation	0.375	$[0.076, 0.604]$
Kendall correlation	0.221	$[-0.016, 0.442]$
Quadratic-weighted κ	0.372	$[0.073, 0.581]$
Mean absolute error	0.771	Not reported

Table 8: The two human passes disagree on the same artifacts, making the choice of label view consequential. Mean absolute error uses the original 0–5 rating scale; all estimates use 72 overlapping pairs.

D.2 EXPANDED CONFIGURATION PANEL

The expanded panel compares the arithmetic $v2$ with the pass1-selected geometric form $\mathrm{mono}^{0.4}\mathrm{stab}^{0.6}$. Its averaged-label association is 0.769, with bootstrap interval $[0.34, 0.95]$ (Figure 4). The result concerns 13 configurations on reused validation labels, not 13 independent model families or unseen data. The plotted geometric index is not $\mathrm{TI}_{\mathrm{var}}$.

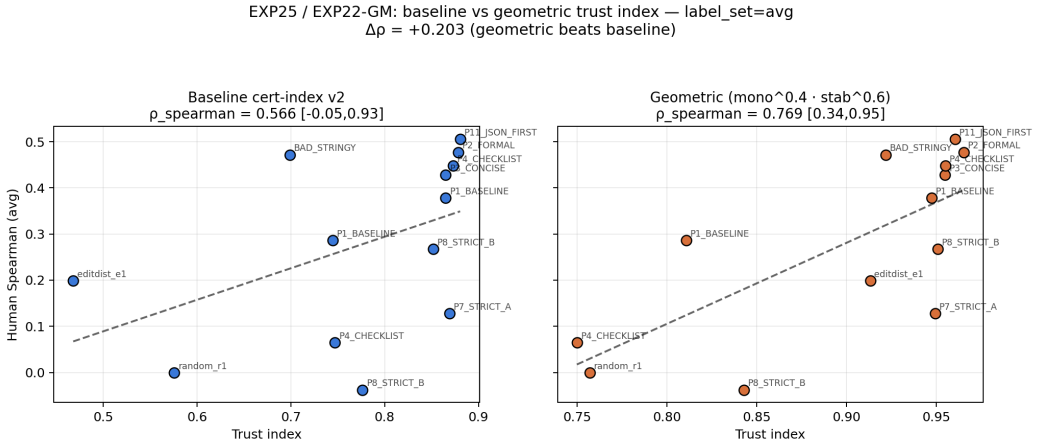


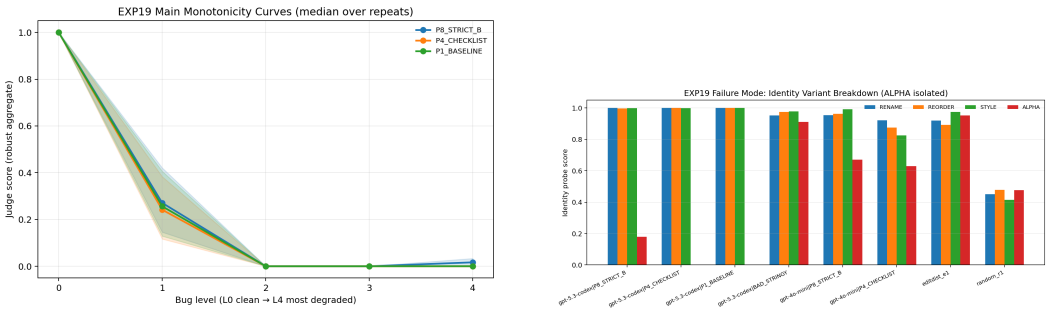
Figure 4: The tuned geometric comparison has a positive association on the expanded validation panel. The two panels compare arithmetic $v2$ with $\mathrm{mono}^{0.4}\mathrm{stab}^{0.6}$ across 13 configurations; the figure retains the archived estimates and intervals.

D.3 NEGATIVE RESULTS

A separate Leanstral panel yields $\mathrm{TI}_{\mathrm{var}}$ associations of -0.1190, -0.0476, and -0.0952 on pass1, pass2, and averaged labels. These recorded results contradict a claim that the fixed index works for every family. They do not establish why the association fails. The different-rubric human pool described in Section 6 also produced no qualifying index in either registered screen. The 32-configuration screen's largest absolute association is negative, so its sign does not support the proposed direction of the screen even before accounting for multiple comparisons.

E PROPERTY DIAGNOSTICS AND PROBE VALIDITY

Historical generator limitations. The recorded experiments use string-edit generators without Lean elaboration checks. The alpha routine rewrites names in the theorem signature, leaves proof-body references unchanged, and can duplicate the `:=` marker. Consequently, its low scores cannot establish failure of invariance under valid alpha-renaming. The renaming and reordering routines also lack checks against variable capture and dependent-binder errors. The bug ladder changes relations and connectives, removes a theorem block, and introduces trivial statements; unavailable edits fall back to comments. Thus cumulative edit level is an intended damage order, not a formally verified ordering of semantic quality. These limitations apply to the archived property statistics throughout the paper.



(a) Codex bug-level means.

(b) Scores by rewrite type.

Figure 5: The archived Codex diagnostics separate cumulative bug edits from unvalidated rewrite probes. The alpha score changes are not evidence of semantic sensitivity because the historical alpha generator can produce malformed text.

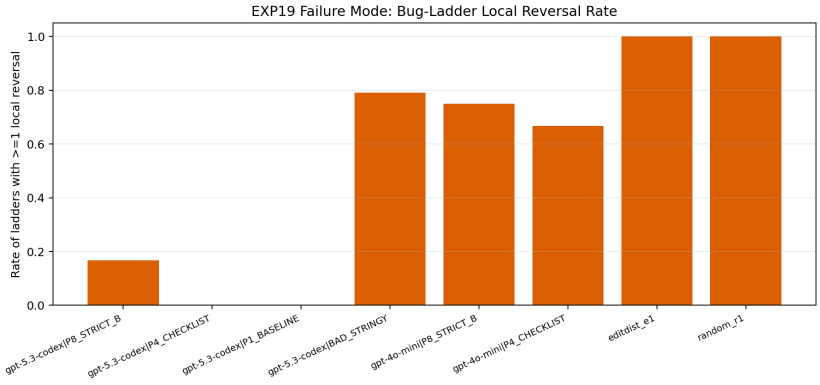


Figure 6: Local increases reveal behavior hidden by the aggregate bug trend. Each rate is the fraction of 24 ladders with at least one increase in the recorded repeat-aggregated score; this figure supplies the reversal counts in Table 3.

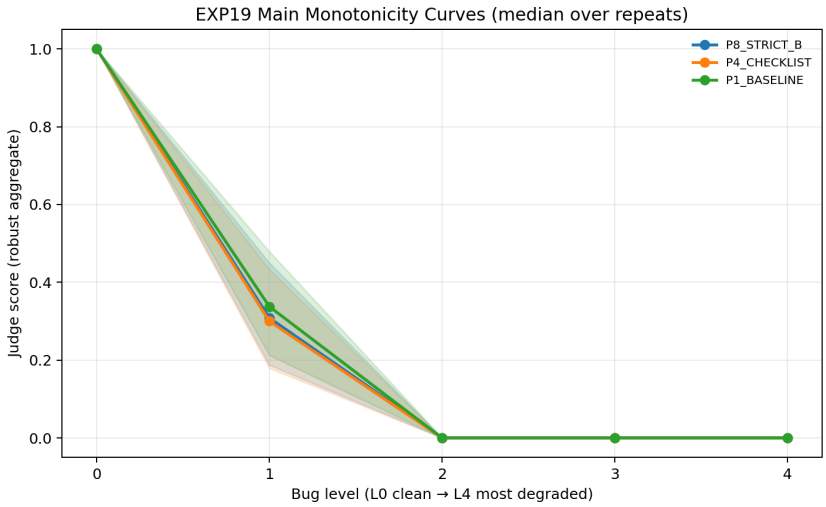


Figure 7: Claude Opus also assigns lower mean scores as historical bug edits accumulate. The plot uses medians over repeats and the same unvalidated string-edit construction; its trend does not establish semantic validity of every ladder.

F Unit Tests as Ground Theorems: A Definition We Do Not Adopt

Section 3 follows common formal-methods usage: the formal specification consists of theorem statements about the implementation, and unit tests are not part of it. We record a competing view that we do not adopt. In Lean 4, a unit test is often itself a closed theorem. The gold `myAdd` artifact states its tests as, e.g., `example : myAdd 1 2 = 3 := by native_decide`, an anonymous theorem with no free variables. On this view, a unit test specifies the implementation at one concrete input, and specifications range from such ground instances to universally quantified properties such as $\forall n$, `myAdd` $n\ 0 = n$. The choice changes terminology, not our measurements. The P3 ladder removes theorem statements and unit tests alike; under the adopted definition, only the removed theorems count as missing specification.